PROJECT CASELET

Online Banking Simulation with Role-Based Access & Transaction PDF Statements

Assigned to: Jaya Suriya K | Location: Coimbatore | Duration: 2 Months

Sl. No.	8	**Duration**	2 Months
Candidate	Jaya Suriya K	**Location**	Coimbatore

PROJECT OVERVIEW

A simulated internet-banking API allowing customers to view accounts, transfer funds and download transaction statements as PDF, with bank employees/admins managing customer accounts and monitoring transactions under strict access control.

TECHNOLOGY STACK

- Backend Framework: ASP.NET Core Web API (C#)
- Data Access: Entity Framework Core — **Database-First** approach
- Database: Microsoft SQL Server (relational schema with proper foreign-key constraints)
- Authentication: JWT (JSON Web Token) based authentication
- Authorization: **Role-Based Access Control (RBAC)** using ASP.NET Core [Authorize(Roles = "...")] policies
- Validation: Data Annotations / FluentValidation with model-state and business-rule checks
- API Documentation: Swagger / OpenAPI
- Architecture: Layered architecture (Controllers → Services → Repositories → DbContext) with DTOs and AutoMapper

CORE DATA ENTITIES

User (Customer/Employee/Admin), Account, Beneficiary, Transaction, Statement.

FUNCTIONAL REQUIREMENTS

- Fund transfer (self/beneficiary) with balance and daily-limit validation
- Beneficiary management with add/approve workflow before first transfer
- Transaction history filtering by date range, type and amount
- Downloadable PDF account statement generation for a chosen period
- Employee/Admin view for account monitoring, freeze/unfreeze operations
- Full audit logging of all monetary transactions

AUTHENTICATION & ROLE-BASED AUTHORIZATION

- Users must log in via a /api/auth/login endpoint that issues a signed JWT access token (with refresh-token support) after verifying hashed credentials (BCrypt/Identity password hasher).
- Every protected endpoint requires a valid bearer token; unauthenticated requests must return HTTP 401.
- The system implements **Role-Based Authentication/Authorization** with distinct roles relevant to this use case (e.g., Admin, Manager/Staff role, and End-User/Customer role), each restricted to its own set of controllers/actions using [Authorize(Roles = "RoleName")].
- Attempts to access another role's resource (e.g., a customer trying to reach an admin-only endpoint) must return HTTP 403 Forbidden.

- Role and claims information is embedded in the JWT payload so the API can authorize requests without extra database round-trips.
- Admin role has full CRUD access for master/reference data; operational roles are limited to their own scope; end-user roles are limited strictly to self-owned records.

VALIDATION & ERROR HANDLING

- Server-side model validation using Data Annotations and/or FluentValidation on all incoming DTOs (required fields, ranges, formats, file-type/size checks for uploads).
- Consistent API error response format with correlation-friendly error codes and messages; global exception-handling middleware for unhandled errors.
- Business-rule validation enforced at the service layer (e.g., duplicate prevention, state-transition checks, capacity/limit checks specific to this use case).
- Proper HTTP status-code usage: 200/201 for success, 400 for validation errors, 401/403 for auth failures, 404 for missing resources, 409 for conflicts.

EXPECTED DELIVERABLES

- ER diagram / database schema and EF Core migrations (or scaffolded DbContext for Database-First).
- ASP.NET Core Web API solution with layered architecture, DTOs, and Swagger documentation.
- JWT-based authentication module with role management and seeded roles/users.
- Postman collection covering all endpoints including role-restricted scenarios.
- Source code repository with README describing setup, configuration and role credentials for testing.